ZAP by Checkmarx Scanning Report

Site: http://localhost:8080

Generated on seg., 1 jun. 2026 15:01:58

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Nível de Risco	Number of Alerts
Alto	0
Médio	4
Baixo	3
Informativo	2

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	13
Info	Informational	http://localhost:8080	Percentage of responses with status code 2xx	43 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 3xx	52 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 4xx	3 %
Info	Informational	http://localhost:8080	Percentage of endpoints with content type text/css	11 %
			Percentage of	

Info	Informational	http://localhost:8080	endpoints with content type text /html	77 %
Info	Informational	http://localhost:8080	Percentage of endpoints with content type text /plain	11 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method GET	88 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method POST	11 %
Info	Informational	http://localhost:8080	Count of total endpoints	9

Alertas

Nome	Nível de Risco	Number of Instances
Ausência de tokens Anti-CSRF	Médio	1
CSP: Failure to Define Directive with No Fallback	Médio	2
Content Security Policy (CSP) Header Not Set	Médio	Systemic
Missing Anti-clickjacking Header	Médio	Systemic
Cookie No HttpOnly Flag	Baixo	1
O servidor vaza informações por meio dos campos de cabeçalho de resposta HTTP "X-Powered-By"	Baixo	Systemic
X-Content-Type-Options Header Missing	Baixo	Systemic
Session Management Response Identified	Informativo	2
User Controllable HTML Element Attribute (Potential XSS)	Informativo	1

Alert Detail

Médio	Ausência de tokens Anti-CSRF
	Não foram localizados tokens Anti-CSRF no formulário de submissão HTML. Uma falsificação de solicitação entre sites (Cross-Site Request Forgery ou simplesmente CSRF) é um ataque que envolve forçar a vítima a enviar uma solicitação HTTP a um destino alvo sem seu conhecimento ou intenção, a fim de realizar uma ação como a vítima. A causa implícita é a funcionalidade do aplicativo usando ações previsíveis em URLs /formulários, de maneira repetível. A natureza do ataque é que o CSRF explora a confiança

Descrição	que um site tem em um usuário. Em contrapartida, um ataque do tipo Cross-Site Scripting (XSS) explora a confiança que um usuário tem em um site. Como o XSS, os ataques CSRF não são necessariamente entre sites, mas também podem ser. A falsificação de solicitação entre sites também é conhecida por "CSRF", "XSRF", "one-click attack", "session riding", "confused deputy", e "sea surf". Os ataques CSRF são efetivos em várias situações, incluindo: * - A vítima tem uma sessão ativa no site de destino; * - A vítima está autenticada por meio de autenticação HTTP no site de destino; * - A vítima está na mesma rede local do site de destino. O CSRF era usado principalmente para executar ações contra um site-alvo usando os privilégios da vítima, mas técnicas recentes foram descobertas para vazamento de informações obtendo acesso às respostas. O risco de vazamento/divulgação não autorizada de informações aumenta drasticamente quando o site de destino é vulnerável a XSS, porque o XSS pode ser usado como uma plataforma para CSRF, permitindo que o ataque opere dentro dos limites da política de mesma origem.
URL	http://localhost:8080/messages
Node Name	http://localhost:8080/messages
Método	GET
Ataque	
Evidence	<form method="POST" action="/messages" class="form">
Other Info	Nenhum token Anti-CSRF conhecido [anticsrf, CSRFToken, __RequestVerificationToken, csrfmiddlewaretoken, authenticity_token, OWASP_CSRFTOKEN, anoncsrf, csrf_token, _csrf, _csrfSecret, __csrf_magic, CSRF, _token, _csrf_token, _csrfToken] foi encontrado nos seguintes formulários HTML: [Form 1: "author"].
Instances	1
Solution	Fase: Arquitetura e Design. Use uma biblioteca verificada ou framework que não permita que essa vulnerabilidade ocorra, ou forneça construções/implementações que tornem essa vulnerabilidade mais fácil de evitar. Por exemplo, use pacotes anti-CSRF, como o OWASP CSRFGuard. Fase: Implementação. Certifique-se de que seu aplicativo esteja livre de problemas de cross-site scripting (XSS), porque a maioria das defesas CSRF pode ser contornada usando script controlado por invasor. Fase: Arquitetura e Design. Gere um número arbitrário de uso único e exclusivo (ou Nonce = "N" de "number" - número em inglês - e "once" de "uma vez" também em inglês) para cada formulário, coloque o nonce no formulário e verifique-o ao receber o formulário. Certifique-se de que o nonce não seja previsível (CWE-330). Observe que isso pode ser contornado usando XSS. Identifique operações especialmente perigosas. Quando o usuário realizar uma operação perigosa, envie uma solicitação de confirmação separada para garantir que o usuário pretendia realizar aquela operação. Observe que isso pode ser contornado usando XSS. Utilize o controle ESAPI Session Management.

	Este controle inclui um componente para CSRF. Não use o método GET para qualquer solicitação que acione uma mudança de estado. Fase: Implementação. Verifique o cabeçalho HTTP Referer para ver se a solicitação foi originada de uma página esperada. Isso pode interromper funcionalidades legítimas, porque os usuários ou proxies podem ter desativado o envio do Referer por motivos de privacidade.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html
CWE Id	352
WASC Id	9
Plugin Id	10202

Médio	**CSP: Failure to Define Directive with No Fallback**
Descrição	The Content Security Policy fails to define one of the directives that has no fallback. Missing /excluding them is the same as allowing anything.
URL	http://localhost:8080/robots.txt
Node Name	http://localhost:8080/robots.txt
Método	GET
Ataque	
Evidence	default-src 'none'
Other Info	The directive(s): frame-ancestors, form-action is/are among the directives that do not fallback to default-src.
URL	http://localhost:8080/sitemap.xml
Node Name	http://localhost:8080/sitemap.xml
Método	GET
Ataque	
Evidence	default-src 'none'
Other Info	The directive(s): frame-ancestors, form-action is/are among the directives that do not fallback to default-src.
Instances	2
Solution	Ensure that your web server, application server, load balancer, etc. is properly configured to set the Content-Security-Policy header.
Reference	https://www.w3.org/TR/CSP/ https://caniuse.com/#search=content+security+policy https://content-security-policy.com/ https://github.com/HtmlUnit/htmlunit-csp https://web.dev/articles/csp#resource-options
CWE Id	693
WASC Id	15
Plugin Id	10055

Médio	**Content Security Policy (CSP) Header Not Set**
Descrição	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to

	declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://localhost:8080/
Node Name	http://localhost:8080/
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/messages
Node Name	http://localhost:8080/messages
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/profile
Node Name	http://localhost:8080/profile
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/search?q=aula
Node Name	http://localhost:8080/search (q)
Método	GET
Ataque	
Evidence	
Other Info	
Instances	Systemic
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/
CWE Id	693
WASC Id	15
Plugin Id	10038

Médio	**Missing Anti-clickjacking Header**
Descrição	The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://localhost:8080
Node Name	http://localhost:8080
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/
Node Name	http://localhost:8080/
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/messages
Node Name	http://localhost:8080/messages
Método	GET
Ataque	
Evidence	
Other Info	
URL	http://localhost:8080/profile

Node Name	http://localhost:8080/profile
Método	GET
Ataque	
Evidence	
Other Info	
Instances	Systemic
Solution	Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020

Baixo	**Cookie No HttpOnly Flag**
Descrição	A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	Set-Cookie: campus_session
Other Info	
Instances	1
Solution	Ensure that the HttpOnly flag is set for all cookies.
Reference	https://owasp.org/www-community/HttpOnly
CWE Id	1004
WASC Id	13
Plugin Id	10010

Baixo	**O servidor vaza informações por meio dos campos de cabeçalho de resposta HTTP "X-Powered-By"**
Descrição	O servidor da web/aplicativo está vazando informações por meio de um ou mais cabeçalhos de resposta HTTP "X-Powered-By". O acesso a essas informações pode facilitar que os invasores identifiquem outras estruturas/componentes dos quais seu aplicativo da web depende e as vulnerabilidades às quais esses componentes podem estar sujeitos.
URL	http://localhost:8080/profile
Node Name	http://localhost:8080/profile
Método	GET

Ataque	
Evidence	X-Powered-By: Express
Other Info	
URL	http://localhost:8080/robots.txt
Node Name	http://localhost:8080/robots.txt
Método	GET
Ataque	
Evidence	X-Powered-By: Express
Other Info	
URL	http://localhost:8080/sitemap.xml
Node Name	http://localhost:8080/sitemap.xml
Método	GET
Ataque	
Evidence	X-Powered-By: Express
Other Info	
URL	http://localhost:8080/styles.css
Node Name	http://localhost:8080/styles.css
Método	GET
Ataque	
Evidence	X-Powered-By: Express
Other Info	
URL	http://localhost:8080/messages
Node Name	http://localhost:8080/messages ()(author,text)
Método	POST
Ataque	
Evidence	X-Powered-By: Express
Other Info	
Instances	Systemic
Solution	Certifique-se de que seu servidor web, servidor de aplicativos, balanceador de carga, etc. esteja configurado para suprimir cabeçalhos "X-Powered-By".
Reference	https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/01-Information_Gathering/08-Fingerprint_Web_Application_Framework https://www.troyhunt.com/shhh-dont-let-your-response-headers/
CWE Id	497
WASC Id	13
Plugin Id	10037

Baixo	X-Content-Type-Options Header Missing
Descrição	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8080/messages
Node Name	http://localhost:8080/messages
Método	GET
Ataque	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8080/profile
Node Name	http://localhost:8080/profile
Método	GET
Ataque	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8080/search?q=monitoria
Node Name	http://localhost:8080/search (q)
Método	GET
Ataque	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
URL	http://localhost:8080/styles.css
Node Name	http://localhost:8080/styles.css

Método	GET
Ataque	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.
Instances	Systemic
Solution	Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application /web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021

Informativo	**Session Management Response Identified**
Descrição	The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	campus_session
Other Info	cookie:campus_session
URL	http://localhost:8080/account
Node Name	http://localhost:8080/account
Método	GET
Ataque	
Evidence	campus_session
Other Info	cookie:campus_session
Instances	2
Solution	Este é um alerta informativo e não uma vulnerabilidade, portanto não há nada a ser corrigido.
Reference	https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/
CWE Id	
WASC Id	
Plugin Id	10112

Informativo	User Controllable HTML Element Attribute (Potential XSS)
Descrição	This check looks at user-supplied input in query string parameters and POST data to identify where certain HTML attribute values might be controlled. This provides hot-spot detection for XSS (cross-site scripting) that will require further review by a security analyst to determine exploitability.
URL	http://localhost:8080/search?q=monitoria
Node Name	http://localhost:8080/search (q)
Método	GET
Ataque	
Evidence	
Other Info	User-controlled HTML attribute values were found. Try injecting special characters to see if XSS might be possible. The page at the following URL: http://localhost:8080/search?q=monitoria appears to include user input in: a(n) [input] tag [value] attribute The user input found was: q=monitoria The user-controlled value was: monitoria
Instances	1
Solution	Validate all input and sanitize output it before writing to any HTML attributes.
Reference	https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
CWE Id	20
WASC Id	20
Plugin Id	10031